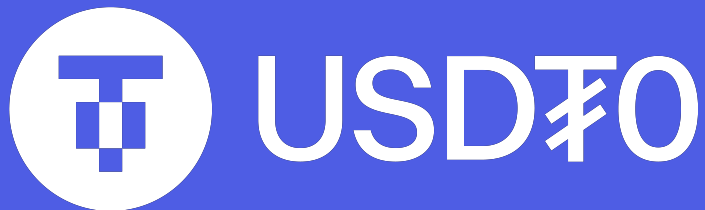


Everdawn USAT Upgrade Review



September 14, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Approach	6
Security Model and Trust Assumptions	6
Privileged Roles	7
Low Severity	8
L-01 Live Third-Party Service Credentials Are Committed to the Repository	8
Notes & Additional Information	8
N-01 CeloOFTEExtension Inheritance Order Silently Determines Storage Layout	8
N-02 Batch Blocklist Functions Are Not Covered by Tests or Formal Verification	9
N-03 Blocklist Helpers Declared private Prevent Composition	10
N-04 Missing NatSpec on Blocklist Functions	10
N-05 EIP-712 Domain Reporting Relies on an Unenforced Override Pairing	11
Conclusion	12
Appendix	13
Issue Classification	13

Summary

Type	Token	Total Issues	6 (5 resolved)
Timeline	From 2026-09-02 To 2026-09-02	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	1 (0 resolved)
		Notes & Additional Information	5 (5 resolved)

Scope

We performed a diff audit of [pull request #110](#) of the [Everdawn-Labs/usdt0-tether-contracts-hardhat](#) repository at commit [5fec6b3](#)).

In scope were the following files:

```
contracts
├─ Tether
│   └─ WithBlockedList.sol
├─ Wrappers
│   ├── ArbitrumExtension.sol
│   ├── OFTEExtension.sol
│   └─ interfaces
│       └─ IERC5267.sol
```

In addition, we verified upgrade safety against the live deployment of [USAT](#) token(i.e. [Celo0FTEExtension](#) contract) on Celo mainnet at address [0xD2ab3C9A02DBBAB236BfEC45D1d755DF4267F771](#).

The purpose of the review was to assess the correctness and safety of the in-place upgrade, the storage compatibility of the new implementation with the live proxy, the associated deployment tooling, and the migration process described in the playbook.

The following were out of scope and were not in place at the time of the review: the LayerZero receive-side security configuration (the DVN set and threshold, the receive library, and the enforced options), the off-chain accounting DVN, the Ethereum-side funding of the lockbox, and the cross-chain peer wiring.

System Overview

The [usdt0-tether-contracts-hardhat](#) repository contains the token implementations behind USDT0 and its sibling assets, which are upgradeable ERC-20 tokens deployed behind OpenZeppelin [TransparentUpgradeableProxy](#) instances across a large number of EVM chains. All implementations share a single inheritance spine. [WithBlockedList](#) provides an owner-controlled address blocklist. [TetherToken](#) adds the token metadata, minting, redemption, and the destruction of blocked funds, and routes the blocklist check through [_beforeTokenTransfer](#). [TetherTokenV2](#) layers gasless approval and transfer flows on top, implementing EIP-2612 [permit](#) and EIP-3009 authorization-based transfers, both of which verify signatures against an EIP-712 domain separator.

Chain-specific behavior is supplied by a family of extension contracts.

[TetherToken0FTEExtension](#) adds the ERC-7802 [crosschainMint](#) and [crosschainBurn](#) entry points that a LayerZero Omnichain Fungible Token adapter calls to move supply between chains, along with an owner-controlled rebranding facility that overrides the token name and symbol. [TetherTokenCeloExtension](#) implements the Celo fee-currency interface, allowing the token to be used to pay transaction gas. [Celo0FTEExtension](#) combines the two. [HyperliquidExtension](#) and [Stable0FTEExtension](#) add trusted-transfer and converter paths respectively. [ArbitrumExtensionV2](#) additionally retains the Arbitrum bridge interface and a one-shot migration routine used to move the legacy Arbitrum deployment onto the USDT0 architecture.

A design characteristic of this family that is central to the audited change is that the EIP-712 domain name is not fixed at initialization. Where the base OpenZeppelin implementation caches a hash of the name in storage, [TetherToken0FTEExtension](#) and [ArbitrumExtensionV2](#) override [_EIP712NameHash](#) to hash the live value returned by [name\(\)](#). Consequently, the domain separator used to verify [permit](#) and EIP-3009 signatures follows any rebranding performed through [updateNameAndSymbol](#).

In this engagement, we reviewed the [pull request](#) making two independent changes. First, it implements EIP-5267 on [TetherToken0FTEExtension](#) and [ArbitrumExtensionV2](#), adding an [eip712Domain](#) function that reports the domain in machine-readable form, advertising the interface through [supportsInterface](#), and emitting an [EIP712DomainChanged](#) event whenever the name is updated. The interface itself is declared in the new [IERC5267](#) file. This allows wallets and other integrators to discover the current

domain without hardcoding assumptions or relying on out-of-band coordination during a rebrand. Second, it refactors `WithBlockedList` so that the single-address blocklist operations delegate to `private` helpers, and adds `addMultipleToBlockedList` and `removeMultipleFromBlockedList` so that many addresses can be blocked or unblocked in one transaction.

Approach

OpenZeppelin reviewed the migration process, the new token implementation, the supporting deployment tooling, and the changes to the OFT contract. The following was verified directly:

- The runtime bytecode of the implementation is currently deployed on Celo, at [0xD2ab3C9A02DBBAB236BfEC45D1d755DF4267F771](#), against the changes in the [pull request #110](#).
- The storage layout of the new implementation, `TetherToken0FTExtension`, is a strict append to the deployed layout: the existing variables keep their slots with no new variables being added.
- The OpenZeppelin Upgrades manifest at [.openzeppelin/usat/celo.json](#), which the upgrade tooling uses as its validation baseline, records a proxy, administrator, implementation address, and storage layout that match the on-chain state.
- The standard ERC-20 interface and its behavior are preserved, including existing allowances, permit nonces, and the EIP-712 domain separator, so existing integrations that rely on the standard interface are unaffected.

Security Model and Trust Assumptions

The security model of this codebase is unchanged by the audited pull request, and remains as established in the previous engagements. The token `owner` is a highly privileged and fully trusted role. It can mint and redeem supply, add and remove addresses from the blocklist, destroy the balances of blocked addresses, rebrand the token, designate the ERC-7802 cross-chain caller, and designate the Celo fee-currency wrapper. The `ProxyAdmin` owner is likewise fully trusted, as it can replace the implementation entirely. Both roles are held by multi-

signature wallets in the recorded deployments. Throughout this review, both roles were assumed to be honest and competent, and no finding is reported that depends on either acting maliciously.

Beyond those roles, the address configured as `oftContract` is trusted to mint and burn supply faithfully on behalf of the cross-chain messaging layer, and on Celo the protocol virtual machine and the configured fee-currency wrapper are trusted to invoke `debitGasFees` and `creditGasFees` with consistent amounts. The correctness of the LayerZero messaging layer itself was outside the scope of this review.

This engagement confirmed deployment against Celo deployment only as requested by the client. The changes in the `ArbitrumExtension` contract were reviewed line-by-line without a corresponding verification against the live Arbitrum deployment.

Privileged Roles

At the time of the review, the token and its proxy administrator were both owned by the same account, a MultiSig wallet at `0x8caD90b6D336930881cD43657C9e0f28FcCBE1BC` configured to require three of five signers. This account can upgrade the token implementation through the proxy administrator and can call every owner-restricted token function, including minting, redeeming, managing the blocklist, connecting the OFT contract, and transferring ownership.

Low Severity

L-01 Live Third-Party Service Credentials Are Committed to the Repository

Around a dozen live third-party credentials are committed to tracked source: RPC access tokens and block-explorer API keys in the `networks` and `etherscan` sections of `hardhat.config.js`, two of them behind a silent `process.env` fallback, plus an Alchemy archive endpoint that embeds its key in both `0FTExtension.test.js` and `ArbitrumExtension.test.js`. No private keys or mnemonics are exposed.

Anyone able to read the repository or its history can consume the team's quota on these services, and the explorer keys additionally permit submitting contract verifications under the team's identity.

Consider rotating every affected credential, since rotation is the only effective remediation once a value has been pushed, then moving each to a `process.env` reference with no hardcoded fallback and adding a secret-scanning step to the workflow.

Update: Acknowledged, not resolved. The Everdawn Labs team stated:

| *RPC are IP gated, we do not mind usage of our free Etherscan Keys.*

Notes & Additional Information

N-01 `Celo0FTExtension` Inheritance Order Silently Determines Storage Layout

The `Celo0FTExtension` contract inherits `TetherToken0FTExtension` followed by `TetherTokenCeloExtension`. Because the latter reserves no gap slots, that declaration order alone determines where each contract's variables land. Compiling the same two bases in the opposite order succeeds without warning and produces `feeCurrencyWrapper` at slot

356 and `oftContract` at slot 357, so the two variables exchange storage regions with no reserved space between them.

The current order is correct and matches what the deployment manifest records for the live implementation. However, nothing in the source signals that the order is load-bearing, and reordering a base list is the kind of change that a reviewer would ordinarily treat as cosmetic. Moreover, it is important to document that both `TetherTokenCeloExtension` and `Celo0FTExtension` are completely different tokens and that no proxy implementing `TetherTokenCeloExtension` should be upgraded to a `Celo0FTExtension` type implementation due to potential storage collision.

Consider adding a comment on the contract declaration stating that the order of the inherited contracts is storage-critical and must not be changed, and clarifying that `TetherTokenCeloExtension` and `Celo0FTExtension` are separate token implementations that must not be used as upgrade targets for one another.

Update: Resolved in [pull request #115](#). The Everdawn Labs team stated:

| *We have added appropriate comments.*

N-02 Batch Blocklist Functions Are Not Covered by Tests or Formal Verification

The new `addMultipleToBlockedList` and `removeMultipleFromBlockedList` both mutate `isBlocked`, and neither is reached by either of the repository's verification mechanisms. Two rules in `TetherTokenFull.spec` are quantified over every method and permit a single selector each, asserting that `isBlocked` may only be set through `addToBlockedList` and cleared through `removeFromBlockedList`. Both rules now admit a counterexample, and both are re-used verbatim by five further [specifications](#). The `onlyOwnerModified` definition gating the non-owner-must-revert rule omits both functions wherever it is declared, which produces no failure but silently narrows the guarantee, and `properties.md` is now inaccurate.

No tracked test references either function, and [continuous integration](#) runs only compilation, tests, and coverage, but not formal verification. Consider extending both rules' selector sets, adding the new selectors to every `onlyOwnerModified` declaration, introducing rules for owner-gating and for equivalence with the single-address paths, updating `properties.md`, and adding unit tests. Running the prover in continuous integration would keep this class of drift from recurring silently.

Update: Resolved in [pull request #111](#). The Everdawn Labs team stated:

| Tests were added in a separate PR.

N-03 Blocklist Helpers Declared `private` Prevent Composition

The extracted `__addToBlockedList` and `__removeFromBlockedList` helpers are declared `private` rather than `internal`. Because `private` members are not inherited, no derived contract can reach them. A review of every inheritor, including the archived implementations under `contracts/Tether/VersionHistory`, confirms that nothing currently calls either helper or the public wrappers internally, so no existing behavior is affected.

The restriction nonetheless removes a natural extension point. A future extension that needs to compose blocklist writes under a different authority, such as a compliance role, an automatic blocking hook, or an operation that blocks and burns atomically, would be forced to route through the `onlyOwner` public wrapper and could not do so.

Consider declaring both helpers `internal`. This preserves the current encapsulation from external callers while leaving the composition path open.

Update: Resolved in [pull request #115](#). The Everdawn Labs team stated:

| We've made both the helpers `internal virtual`.

N-04 Missing NatSpec on Blocklist Functions

None of the four public functions of the `WithBlockedList` contract carry NatSpec documentation, and the only documented member of the file is the `onlyNotBlocked` modifier. The two newly added batch functions inherit this omission. Documentation coverage is inconsistent across the repository, so this is not a regression, but these four functions constitute the privileged compliance surface that a multi-signature signer reviews before approving a transaction.

Consider adding `@notice` and `@param` tags to all four functions, together with a `@dev` note recording that duplicate and already-blocked addresses are accepted and cause the corresponding event to be re-emitted.

Update: Resolved in [pull request #115](#). The Everdawn Labs team stated:

| *We have added appropriate NatSpec.*

N-05 EIP-712 Domain Reporting Relies on an Unenforced Override Pairing

The new `eip712Domain` function on `TetherToken0FTEExtension` and `ArbitrumExtensionV2` reports `this.name()` as the domain name. That is correct only because both contracts also override `_EIP712NameHash` to hash the live `name()` instead of the value cached in storage at initialization. The pairing is load-bearing in production: on the Arbitrum USDT0 proxy the cached hash remains `keccak256("Tether USD")` while `name()` returns `USD₭0`, and the live `DOMAIN_SEPARATOR` corresponds to the latter.

Nothing enforces that the two overrides stay paired, and `Celo0FTEExtension` shows how readily they could drift, since diamond inheritance obliges it to redeclare both. A future extension overriding `name` but omitting `_EIP712NameHash` would compile cleanly, keep reporting a plausible domain, and silently break every gasless approval built from it. The accompanying tests assert the individual field values but never that those fields reproduce the contract's own `DOMAIN_SEPARATOR`, which is the only property determining whether an integrator's signature is accepted, and no test covers the four inheriting variants.

Consider documenting the invariant on both implementations that `_EIP712NameHash` is dependent on `name()` rather than reading the cached `_HASHED_NAME`, and adding tests with a recomputation of the domain separator checked against `DOMAIN_SEPARATOR` across every deployable extension.

Update: Resolved in [pull request #115](#). The Everdawn Labs team stated:

| *We have added comments.*

Conclusion

The audited changes in the pull request are small and well executed. The new entry points are either owner-facing or view functions supporting EIP-5267, which lets integrators retrieve the EIP-712 domain rather than hardcode it. No critical, high or medium severity issues were identified. The changes provide new view functionalities to fetch EIP-712 domain to create signatures and support for batch blocklisting operations for the owner.

The change adds no state and alters no storage slot, selector, or event, and the upgrade of the live Celo deployment named in scope is safe, with the evidence recorded. One low severity and few note severity issues were identified primarily focused on strengthening the codebase.

We thank the Everdawn Labs team for their responsiveness and collaboration throughout this review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.